

《移动终端软件开发技术》

Mobile Application Development

6. Android数据共享

谢坤
计算机学院（国家示范性软件学院）

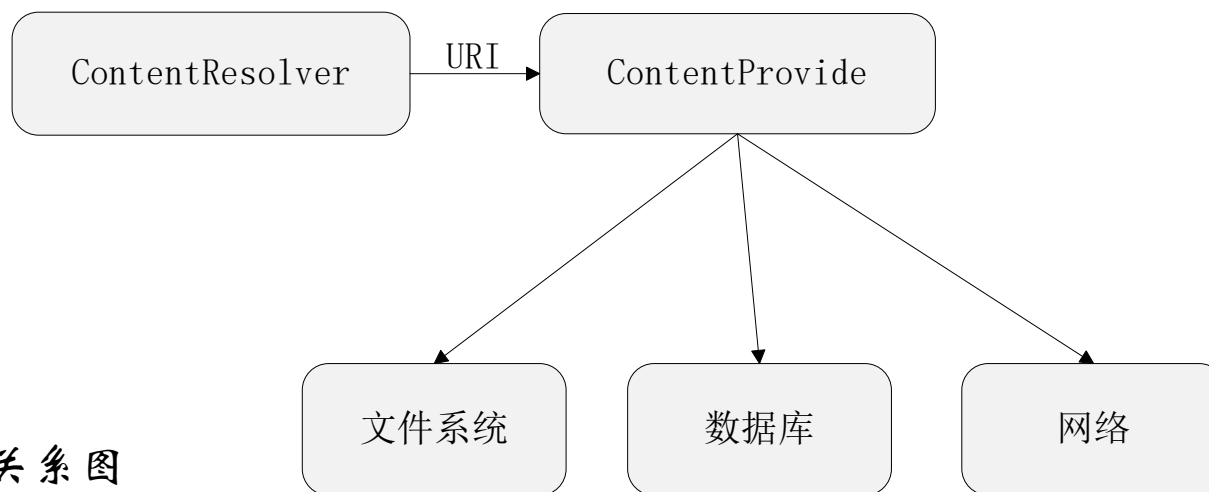
1 ContentProvider简介

1 ContentProvider简介

- ContentProvider（数据提供者）：
 - 应用程序之间共享数据的一种接口机制
 - 可以指定需要共享的数据
 - 其它应用程序可在不知数据来源、路径的情况下对共享数据进行查询、添加、删除和更新等操作
 - Android系统内置的数据也可通过ContentProvider提供给用户使用

1 ContentProvider简介

- ContentProvider:
 1. 底层的数据源的实现
 - 数据库、文件系统或网络等数据源
 2. 实现数据操作的ContentProvider接口函数继承
 - 添加、删除、查找和更新等功能
 - 接口函数不可直接调用，需使用ContentResolver对象，通过URI间接调用



调用关系图

1 ContentProvider简介

- ContentProvider:
 - 通过URI确定要访问的ContentProvider数据集
 - 发起一个请求:
 1. Android系统根据URI确定处理这个查询的ContentProvider
 2. 初始化ContentProvider所有需要的资源(Android系统完成)
- 只有一个ContentProvider对象，同时可与多个ContentResolver交互

1 ContentProvider简介

- ContentProvider:
 - 完全屏蔽数据提供组件的数据存储方法
 - 提供标准数据操作接口
 - 对数据使用者屏蔽内部数据存储方法
 - 对数据使用者屏蔽数据存储设备
 - ContentProvider使用方法的简化
 - 只需调用接口函数完成数据操作
 - 存储方法交由ContentProvider设计者考虑

1 ContentProvider简介

- ContentProvider:
 - ContentProvider数据集:
 - 类似于数据库的数据表
 - 每行一条记录，每列具有相同的数据类型
 - 每条记录包含一个长整型字段“_ID”
 - 用来唯一标识每条记录
 - ContentProvider可提供多个数据集，调用者使用URI对不同数据集的数据进行操作

ContentProvider数据集示例

_ID	NAME	AGE	HEIGHT
1	Tom	21	1.81
2	Jim	22	1.78

1 ContentProvider简介

- ContentProvider:

- ContentProvider使用的URI语法结构:

1 content://<authority>/<data_path>/<id>

- content://: 通用前缀, 表示该URI用于ContentProvider定位资源, 无需修改
- <authority>: 授权者名称, 用来确定具体由哪一个ContentProvider提供资源
 - <authority>一般采用应用包名的方式进行命名, 以保证唯一性。

1 ContentProvider简介

- ContentProvider:

1 content://<authority>/<data_path>/<id>

- ContentProvider使用的URI语法结构:

- <data_path>: 数据路径, 用来确定请求的是哪个数据集
 - 若仅提供一个数据集, 数据路径可省略
 - 若提供多个数据集, 数据路径必须指明具体的某个数据集
 - 数据集的数据路径可以写成多段格式, 例如people/girl和people/boy。
- <id>: 数据编号, 通过匹配数据集中_ID字段的值来唯一确定数据集中的
一条记录
 - 若请求的数据不只限于一条, 则<id>可省略, 例如
 - 请求整个people/boy数据集
 1. content://com.example.app.provider/people/boy
 - 请求people/boy数据集中第3条数据
 2. content://com.example.app.provider/people/boy/3

2 运行时权限

2.1 Android权限机制详解

1. `<manifest xmlns:android="http://schemas.android.com/apk/res/android"`
2. `package="com.example.broadcasttest">`
3. `<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />`
4. `...`
5. `</manifest>`



2.1 Android权限机制详解

- Android 6.0系统中新增了运行时权限功能
 - 用户不需要在安装软件的时候一次性授权所有申请的权限
 - 用户可以在软件的使用过程中对某一项权限的申请进行授权
 - 权限分类：
 - 普通权限和危险权限

2.1 Android权限

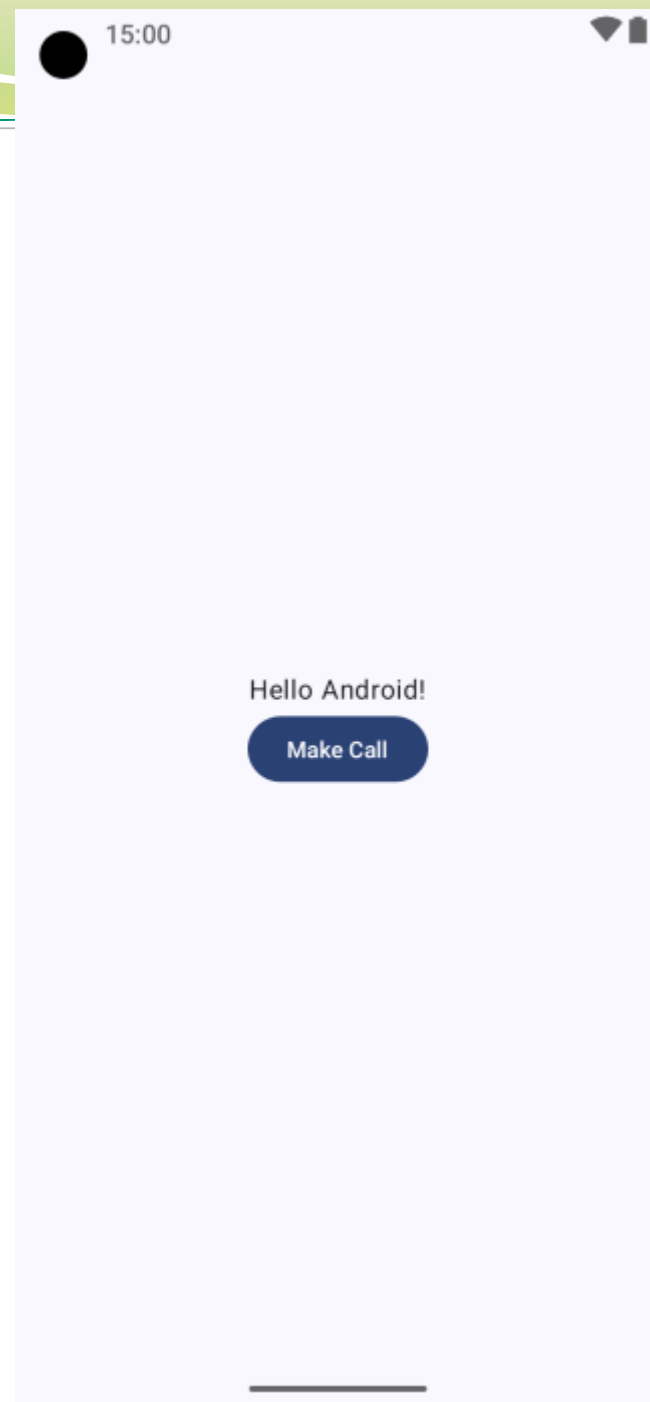
11组30个权限

权限组名	权限名
CALENDAR	READ_CALENDAR
	WRITE_CALENDAR
CALL_LOG	READ_CALL_LOG WRITE_CALL_LOG PROCESS_OUTGOING_CALLS
CAMERA	CAMERA
CONTACTS	READ_CONTACTS
	WRITE_CONTACTS
	GET_ACCOUNTS
LOCATION	ACCESS_FINE_LOCATION
	ACCESS_COARSE_LOCATION ACCESS_BACKGROUND_LOCATION
MICROPHONE	RECORD_AUDIO
PHONE	READ_PHONE_STATE
	READ_PHONE_NUMBERS
	CALL_PHONE
	ANSWER_PHONE_CALLS
	ADD_VOICEMAIL
	USE_SIP ACCEPT_HANDOVER
SENSORS	BODY_SENSORS
ACTIVITY_RECOGNITION	ACTIVITY_RECOGNITION
SMS	SEND_SMS
	RECEIVE_SMS
	READ_SMS
	RECEIVE_WAP_PUSH
	RECEIVE_MMS
STORAGE	READ_EXTERNAL_STORAGE
	WRITE_EXTERNAL_STORAGE
	ACCESS_MEDIA_LOCATION

2.2 程序运行时申请权限

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (.....) {
        .....
        Button(
            onClick = { makeCall(activity) }
        ) { Text(stringResource(R.string.button_call)) }
    }
}

private fun makeCall(activity: Activity?) {
    try {
        val intent = Intent(Intent.ACTION_CALL)
        intent.data = Uri.parse("tel:10086")
        activity?.startActivity(intent)
    } catch (e: SecurityException) {
        e.printStackTrace()
    }
}
```



2.2 程序运行时申请权限

```
<uses-feature
    android:name="android.hardware.telephony"
    android:required="false" />
<uses-permission
    android:name="android.permission.CALL_PHONE" />
```

```
java.lang.SecurityException: Permission Denial: starting Intent { act=android.intent.action.CALL
    at android.os.Parcel.createException(Parcel.java:2069)
    at android.os.Parcel.readException(Parcel.java:2037)
    at android.os.Parcel.readException(Parcel.java:1986)
    at android.app.IActivityTaskManager$Stub$Proxy.startActivity(IActivityTaskManager.java:3827)
    at android.app.Instrumentation.execStartActivity(Instrumentation.java:1705)
    at android.app.Activity.startActivityForResult(Activity.java:5173)
    at androidx.fragment.app.FragmentActivity.startActivityForResult(FragmentActivity.java:767)
    at android.app.Activity.startActivityForResult(Activity.java:5131)
    at androidx.fragment.app.FragmentActivity.startActivityForResult(FragmentActivity.java:754)
    at android.app.Activity.startActivity(Activity.java:5502)
    at android.app.Activity.startActivity(Activity.java:5470)
    at com.example.runtimepermissiontest.MainActivity$onCreate$1.onClick(MainActivity.kt:19)
```

2.2 程序运行时申请权限

```
@Composable
fun Greeting(
    requestPermissionLauncher: ActivityResultLauncher<String>,
    .....
) {
    val context = LocalContext.current

    Column (.....) {
        .....
        Button(
            onClick = { requestCallPermission(context, requestPermissionLauncher) }
        ) { Text(stringResource(R.string.button_call)) }
    }
}
```

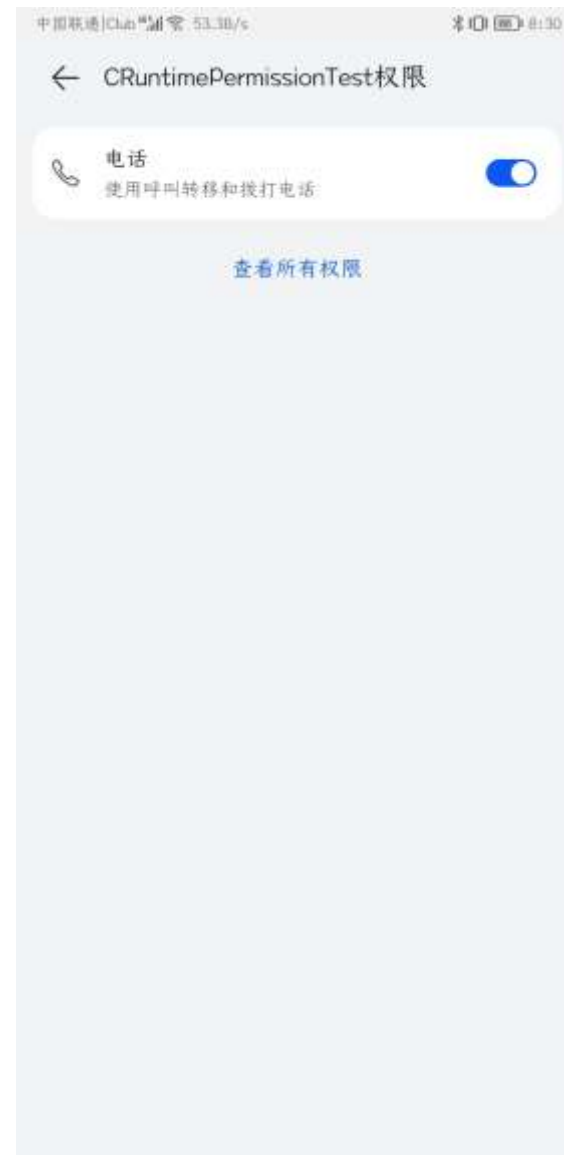
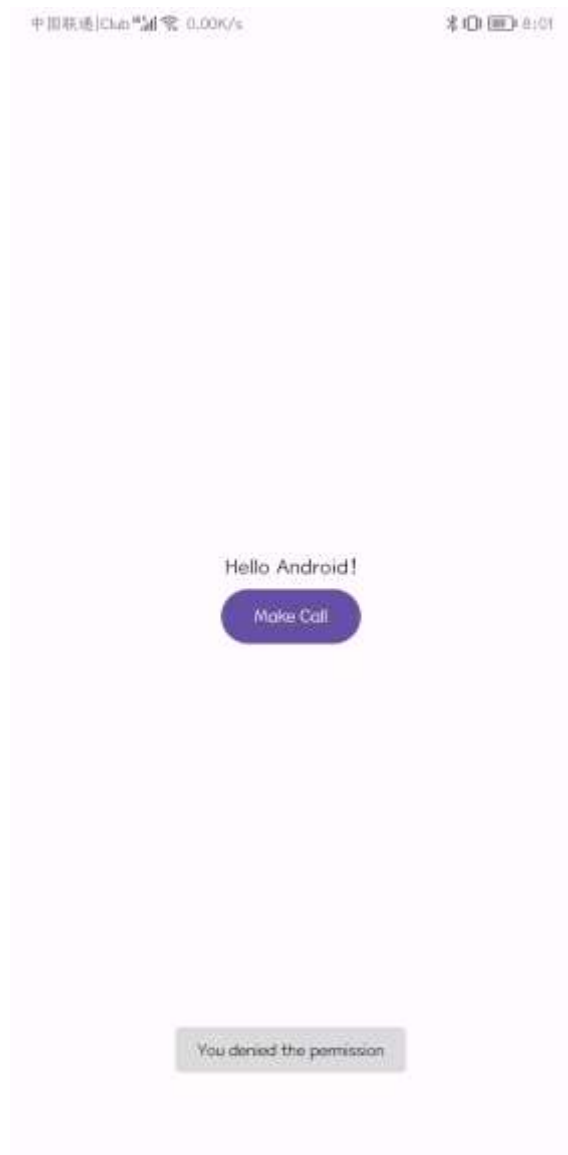
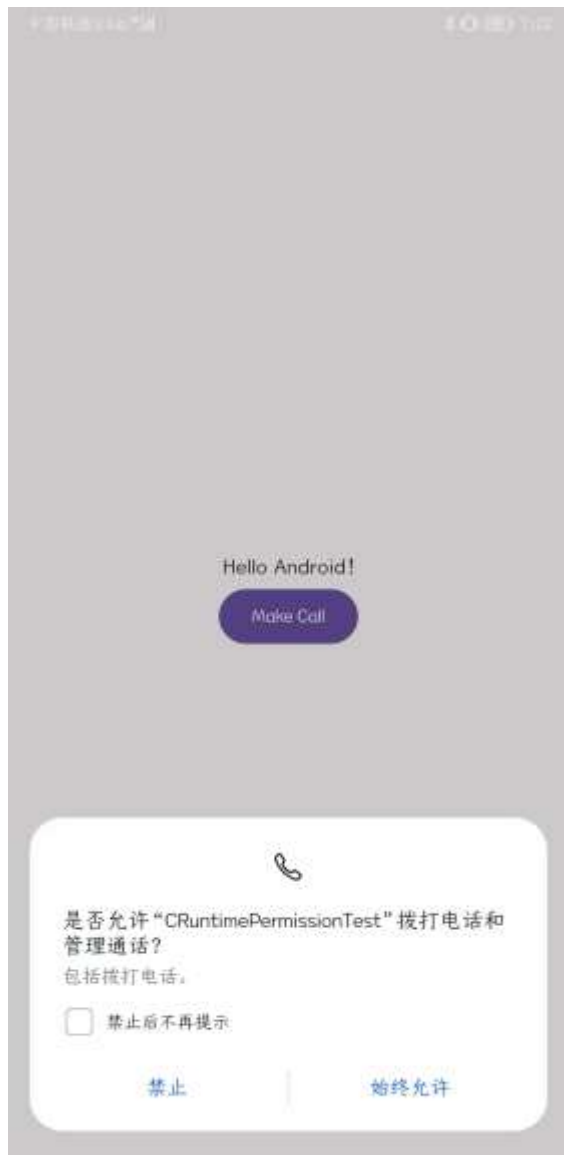
```
private fun requestCallPermission(
    context: Context,
    requestPermissionLauncher: ActivityResultLauncher<String>
) {
    when {
        ContextCompat.checkSelfPermission(
            context,
            Manifest.permission.CALL_PHONE
        ) == PackageManager.PERMISSION_GRANTED -> { makeCall(context as Activity) }
        else -> { requestPermissionLauncher.launch(Manifest.permission.CALL_PHONE) }
    }
}
```

```
private fun makeCall(activity: Activity) {
    try {
        val intent = Intent(Intent.ACTION_CALL)
        intent.data = Uri.parse("tel:10086")
        activity.startActivity(intent)
    } catch (e: SecurityException) {
        e.printStackTrace()
    }
}
```


2.2 程序运行时申请权限

```
class MainActivity : ComponentActivity() {  
    private lateinit var requestPermissionLauncher: ActivityResultLauncher<String>  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
  
        requestPermissionLauncher = registerForActivityResult(  
            ActivityResultContracts.RequestPermission()  
        ) { isGranted: Boolean ->  
            if (isGranted) { makeCall(this) }  
            else {  
                Toast.makeText(  
                    this,  
                    "You denied the permission",  
                    Toast.LENGTH_SHORT  
                ).show()  
            }  
        }  
        .....  
    }  
}
```

2.2 程序运行时申请权限



3 访问其他程序中的数据

3.1 ContentResolver的基本用法

- ContentResolver:
 - 通过ContentResolver对象和URI进行数据操作
 - insert()、update()、delete()、query()
 - 基于URI和数据集的数据格式进行数据操作
 - 每个Android组件都具有一个ContentResolver对象
 - 获取ContentResolver对象的方法:
 - 调用getContentResolver()方法

3.1 ContentResolver的基本用法

- ContentResolver:
 - 将内容URI字符串解析成Uri对象: Uri.parse() 方法

```
1 val uri = Uri.parse("content://com.example.app.provider/people/boy")
```

```
1. val cursor = contentResolver.query(  
2.     uri,  
3.     projection,  
4.     selection,  
5.     selectionArgs,  
6.     sortOrder)
```

3.1 ContentResolver的基本用法

- ContentResolver: query() 方法的参数说明

query()方法参数	对应SQL部分	描述
uri	from table_name	指定查询某个应用程序下的某一张表
projection	select column1, column2	指定查询的列名
selection	where column = value	指定where的约束条件
selectionArgs	-	为where中的占位符提供具体的值
sortOrder	order by column1, column2	指定查询结果的排序方式

```
1. while (cursor.moveToNext()) {  
2.     val column1 = cursor.getString(cursor.getColumnIndex("name"))  
3.     val column2 = cursor.getInt(cursor.getColumnIndex("age"))  
4. }  
5. cursor.close()
```

3.1 ContentResolver的基本用法

1. `val values = contentValuesOf("name" to "Tom", "age" to 5)`
2. `contentResolver.insert(uri, values)`

1. `val values = contentValuesOf("name" to "")`
2. `contentResolver.update(uri, values, "name = ? and age = ?", arrayOf("Tom", "5"))`

1. `contentResolver.delete(uri, "age = ?", arrayOf("5"))`

3.2 读取系统联系人

```
private fun readContacts(context: Context, contactsList: MutableList<String>) {  
    // 查询联系人数据  
    context.contentResolver.query(  
        ContactsContract.CommonDataKinds.Phone.CONTENT_URI,  
        null, null, null, null  
   )?.apply {  
        while (moveToNext()) {  
            // 获取联系人姓名  
            val displayName = getString(  
                getColumnIndexOrThrow(ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME)  
            )  
            // 获取联系人手机号  
            val number = getString(  
                getColumnIndexOrThrow(ContactsContract.CommonDataKinds.Phone.NUMBER)  
            )  
            contactsList.add("$displayName\n$number")  
        }  
        close()  
    }  
}
```

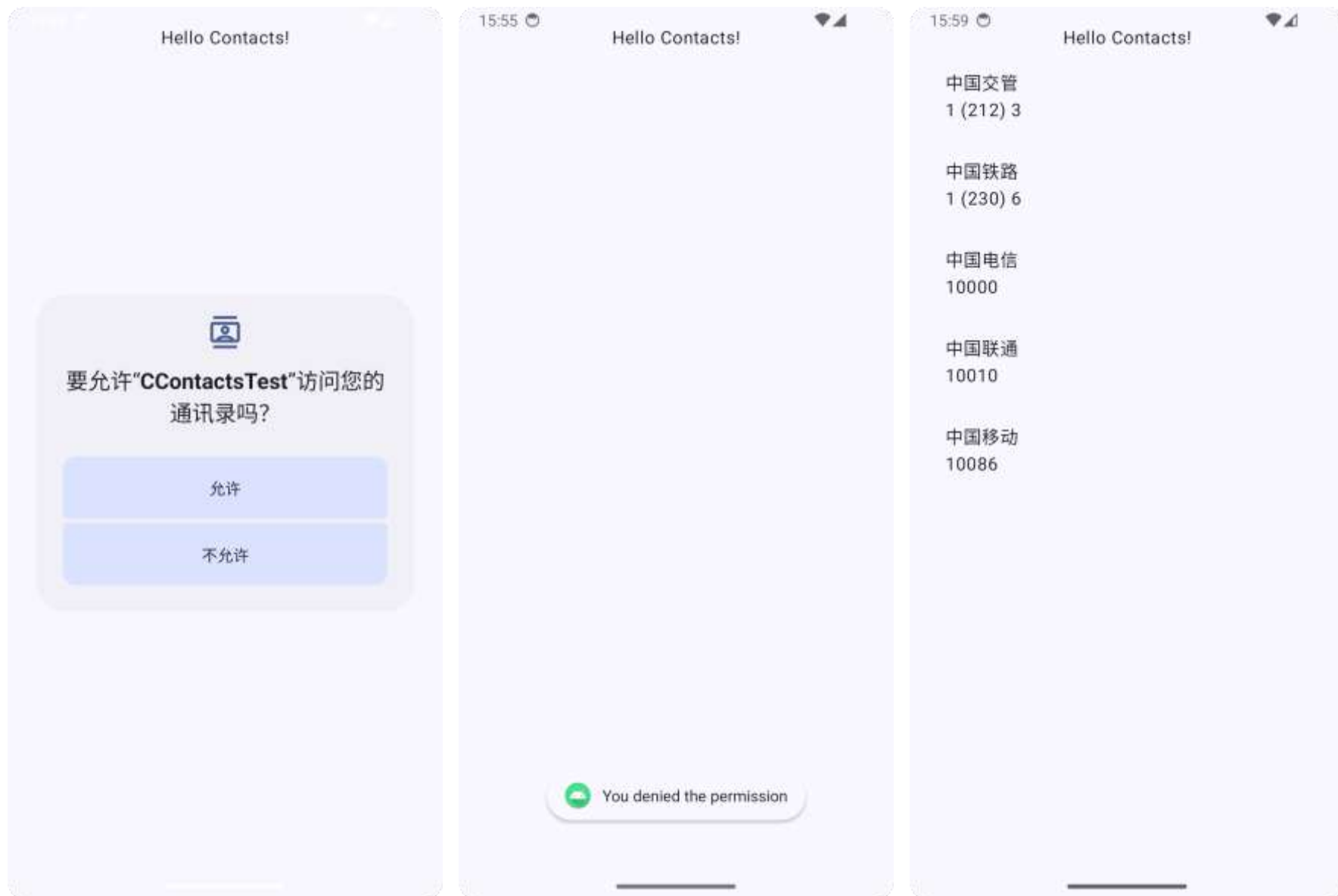


3.2 读取 系统联系人

```
<uses-permission android:name=  
"android.permission.READ_CONTACTS"  
>
```

```
class MainActivity : AppCompatActivity() {  
  
    private lateinit var requestPermissionLauncher: ActivityResultLauncher<String>  
    private val contactsList = mutableListOf<String>()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
  
        super.onCreate(savedInstanceState)  
  
        requestPermissionLauncher = registerForActivityResult(  
            ActivityResultContracts.RequestPermission()  
        ) { isGranted: Boolean ->  
            if (isGranted) { readContacts(this, contactsList) }  
            else {  
                Toast.makeText(this, "You denied the permission", Toast.LENGTH_SHORT).show()  
            }  
        }  
  
        when {  
            ContextCompat.checkSelfPermission(  
                this,  
                Manifest.permission.READ_CONTACTS  
            ) == PackageManager.PERMISSION_GRANTED -> { readContacts(this, contactsList) }  
            else -> { requestPermissionLauncher.launch(Manifest.permission.READ_CONTACTS) }  
        }  
  
        .....  
        setContentView {  
            .....  
            Greeting(  
                modifier = Modifier.padding(16.dp),  
                contactsList  
            )  
            .....  
        }  
    }  
}
```

3.2 读取系统联系人



4 创建自己的ContentProvider

4.1 创建ContentProvider的步骤

- 创建ContentProvider:
 - 通过继承ContentProvider类来创建新的ContentProvider
 - 分为三步:
 1. 继承ContentProvider，并重载六个函数
 2. 声明内容URI，实现UriMatcher
 3. 注册ContentProvider

4.1 创建ContentProvider的步骤

```
1. class MyProvider : ContentProvider() {  
2.     override fun onCreate(): Boolean {  
3.         return false  
4.     }  
5.     override fun query(uri: Uri, projection: Array<String>?, selection: String?, selectionArgs: Array<String>?, sortOrder: String?): Cursor? {  
6.         return null  
7.     }  
8.     override fun insert(uri: Uri, values: ContentValues?): Uri? {  
9.         return null  
10.    }  
11.    override fun update(uri: Uri, values: ContentValues?, selection: String?, selectionArgs: Array<String>?): Int {  
12.        return 0  
13.    }  
14.    override fun delete(uri: Uri, selection: String?, selectionArgs: Array<String>?): Int {  
15.        return 0  
16.    }  
17.    override fun getType(uri: Uri): String? {  
18.        return null  
19.    }  
20. }
```

4.1 创建ContentProvider的步骤

- <id>: 匹配数据集中_ID字段, 唯一确定数据集中的一条记录
- 请求整个table1表: `content://com.example.app.provider/table1`
- 请求table1表中某条数据: `content://com.example.app.provider/table1/1`
- 通配符
 - *: 匹配任意长度的任意字符
 1. 匹配任意表的URI: `content://com.example.app.provider/*`
 - #: 匹配任意长度的数字
 2. 匹配表中任意一行数据的URI: `content://com.example.app.provider/table1/#`

4.1 创建ContentProvider的步骤

```
1. class MyProvider : ContentProvider() {  
2.     private val table1Dir = 0  
3.     private val table1Item = 1  
4.     private val table2Dir = 2  
5.     private val table2Item = 3  
  
6.     private val uriMatcher = UriMatcher(UriMatcher.NO_MATCH)  
  
7.     init {  
8.         uriMatcher.addURI("com.example.app.provider", "table1", table1Dir)  
9.         uriMatcher.addURI("com.example.app.provider ", "table1/#", table1Item)  
10.        uriMatcher.addURI("com.example.app.provider ", "table2", table2Dir)  
11.        uriMatcher.addURI("com.example.app.provider ", "table2/#", table2Item)  
12.    }  
13.    ...
```

```
14. override fun query(uri: Uri, projection: Array<String>?,  
15.        selection: String?, selectionArgs: Array<String>?,  
16.        sortOrder: String?): Cursor? {  
17.    when (uriMatcher.match(uri)) {  
18.        table1Dir -> {  
19.            // 查询table1表中的所有数据  
20.        }  
21.        table1Item -> {  
22.            // 查询table1表中的单条数据  
23.        }  
24.        table2Dir -> {  
25.            // 查询table2表中的所有数据  
26.        }  
27.        table2Item -> {  
28.            // 查询table2表中的单条数据  
29.        }  
30.    }  
31.    ...  
32. }  
33. ...  
34. }
```

4.1 创建ContentProvider的步骤

- getType() 方法
 - 获取URI对象所对应的MIME类型（MIME字符串）
 - MIME字符串格式：
 - 必须以vnd开头。
 - 如果内容URI以路径结尾，则后接android.cursor.dir/；如果内容URI以id结尾，则后接android.cursor.item/。
 - 最后接上vnd.<authority>.<path>。
 - content://com.example.app.provider/table1:
 1. vnd.android.cursor.dir/vnd.com.example.app.provider.table1
 - content://com.example.app.provider/table1/1:
 2. vnd.android.cursor.item/vnd.com.example.app.provider.table1

4.1 创建ContentProvider的步骤

```
1. class MyProvider : ContentProvider() {  
2.     ...  
3.     override fun getType(uri: Uri) = when (uriMatcher.match(uri)) {  
4.         table1Dir -> "vnd.android.cursor.dir/vnd.com.example.app.provider.table1"  
5.         table1Item -> "vnd.android.cursor.item/vnd.com.example.app.provider.table1"  
6.         table2Dir -> "vnd.android.cursor.dir/vnd.com.example.app.provider.table2"  
7.         table2Item -> "vnd.android.cursor.item/vnd.com.example.app.provider.table2"  
8.         else -> null  
9.     }  
10. }
```

4.2 跨程序数据共享

New Android Component

Content Provider

Creates a new content provider component and adds it to your Android manifest

Class Name

DatabaseProvider

URI Authorities

com.example.cdatasetest.provider

☒ Exported

☒ Enabled

Source Language

Kotlin

Cancel Finish

```
.....  
<permission android:name="com.example.cdatasetest.PROVIDER_PERMISSION"  
    android:protectionLevel="normal"/>  
  
<application  
    .....  
    <provider  
        android:name=".DatabaseProvider"  
        android:authorities="com.example.cdatasetest.provider"  
        android:enabled="true"  
        android:exported="true"/>  
    .....  
</application>  
  
</manifest>
```

4.2 跨程序数据共享

```
class DatabaseProvider : ContentProvider() {

    private val bookDir = 0
    private val bookItem = 1
    private val categoryDir = 2
    private val categoryItem = 3
    private val authority = "com.example.cdatabasetest.provider"
    private var dbHelper: MyDatabaseHelper? = null

    private val uriMatcher by lazy {
        val matcher = UriMatcher(UriMatcher.NO_MATCH)
        matcher.addURI(authority, "book", bookDir)
        matcher.addURI(authority, "book/#", bookItem)
        matcher.addURI(authority, "category", categoryDir)
        matcher.addURI(authority, "category/#", categoryItem)
        matcher
    }

    override fun onCreate() = context?.let {
        dbHelper = MyDatabaseHelper(it, "BookStore.db", 2)
        true
    } ?: false
}
```

```
override fun query(
    uri: Uri, projection: Array<String>?, selection: String?,
    selectionArgs: Array<String>?, sortOrder: String?
) = dbHelper?.let {
    val db = it.readableDatabase
    when (uriMatcher.match(uri)) {
        bookDir -> db.query(
            "Book", projection, selection, selectionArgs,
            null, null, sortOrder
        )
        bookItem -> {
            val bookId = uri.pathSegments[1]
            db.query(
                "Book", projection, "id = ?", arrayOf(bookId),
                null, null, sortOrder
            )
        }
        categoryDir -> db.query(
            "Category", projection, selection, selectionArgs,
            null, null, sortOrder
        )
        categoryItem -> {
            val categoryId = uri.pathSegments[1]
            db.query(
                "Category", projection, "id = ?",
                arrayOf(categoryId), null, null, sortOrder
            )
        }
        else -> null
    }
}
```

4.2 跨 程序 数据 共享

```
override fun insert(uri: Uri, values: ContentValues?) = dbHelper?.let {
    it.writableDatabase.use { db ->
        val uriReturn = when (uriMatcher.match(uri)) {
            bookDir, bookItem -> {
                val newBookId = db.insert("Book", null, values)
                Uri.parse("content://$authority/book/$newBookId")
            }
            categoryDir, categoryItem -> {
                val newCategoryId = db.insert("Category", null, values)
                Uri.parse("content://$authority/category/$newCategoryId")
            }
            else -> null
        }
        uriReturn?.let { returnUri -> context?.contentResolver?.notifyChange(returnUri, null) }
        uriReturn
    }
}

override fun update(uri: Uri, values: ContentValues?, selection: String?, selectionArgs: Array<String>?) = dbHelper?.let {
    it.writableDatabase.use { db ->
        val updatedRows = when (uriMatcher.match(uri)) {
            bookDir -> db.update("Book", values, selection, selectionArgs)
            bookItem -> {
                val bookId = uri.pathSegments[1]
                db.update("Book", values, "id = ?", arrayOf(bookId))
            }
            categoryDir -> db.update("Category", values, selection, selectionArgs)
            categoryItem -> {
                val categoryId = uri.pathSegments[1]
                db.update("Category", values, "id = ?", arrayOf(categoryId))
            }
            else -> 0
        }
        if (updatedRows > 0) {
            context?.contentResolver?.notifyChange(uri, null)
        }
        updatedRows
    }
} ?: 0
```

4.2 跨程序数据共享

```
override fun delete(uri: Uri, selection: String?, selectionArgs: Array<String>?) = dbHelper?.let {
    it.writableDatabase.use { db ->
        val deletedRows = when (uriMatcher.match(uri)) {
            bookDir -> db.delete("Book", selection, selectionArgs)
            bookItem -> {
                val bookId = uri.pathSegments[1]
                db.delete("Book", "id = ?", arrayOf(bookId))
            }
            categoryDir -> db.delete("Category", selection, selectionArgs)
            categoryItem -> {
                val categoryId = uri.pathSegments[1]
                db.delete("Category", "id = ?", arrayOf(categoryId))
            }
            else -> 0
        }
        if (deletedRows > 0) {
            context?.contentResolver?.notifyChange(uri, null)
        }
        deletedRows
    }
} ?: 0

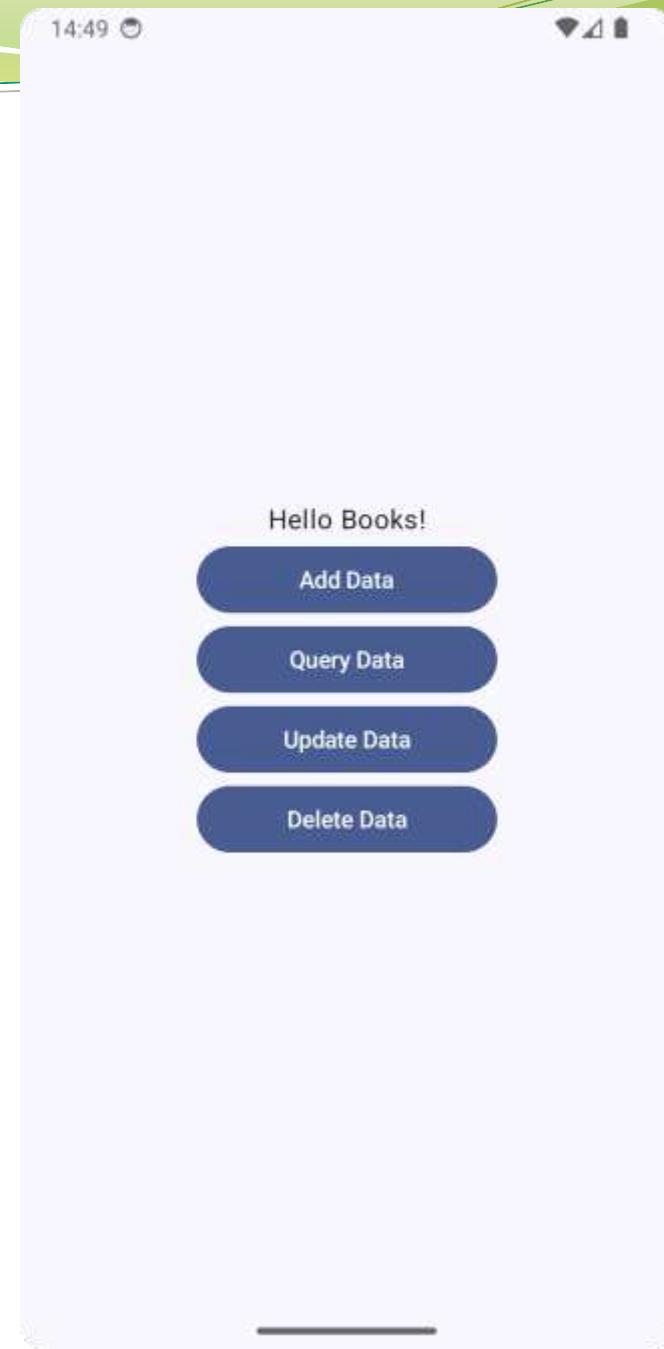
override fun getType(uri: Uri) = when (uriMatcher.match(uri)) {
    bookDir -> "vnd.android.cursor.dir/vnd.com.example.cdatabasetest.provider.book"
    bookItem -> "vnd.android.cursor.item/vnd.com.example.cdatabasetest.provider.book"
    categoryDir -> "vnd.android.cursor.dir/vnd.com.example.cdatabasetest.provider.category"
    categoryItem -> "vnd.android.cursor.item/vnd.com.example.cdatabasetest.provider.category"
    else -> null
}
```

4.2 跨程序数据共享

```
<uses-permission android:name="com.example.cdbasetest.PROVIDER_PERMISSION"/>
```

```
@Composable
fun Greeting(modifier: Modifier = Modifier) {
    val context = LocalContext.current
    var bookId by remember { mutableStateOf<String?>(null) }

    Column(.....) {
        Text(text = "Hello Books!")
        Button(
            onClick = { bookId = addData(context) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.add_data))}
        Button(
            onClick = { queryData(context) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.query_data))}
        Button(
            onClick = { updateData(bookId, context) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.update_data))}
        Button(
            onClick = { deleteData(bookId, context) },
            modifier = Modifier.fillMaxWidth(0.5f)
        ) {Text(stringResource(R.string.delete_data))}
    }
}
```



4.2 跨程序数据共享

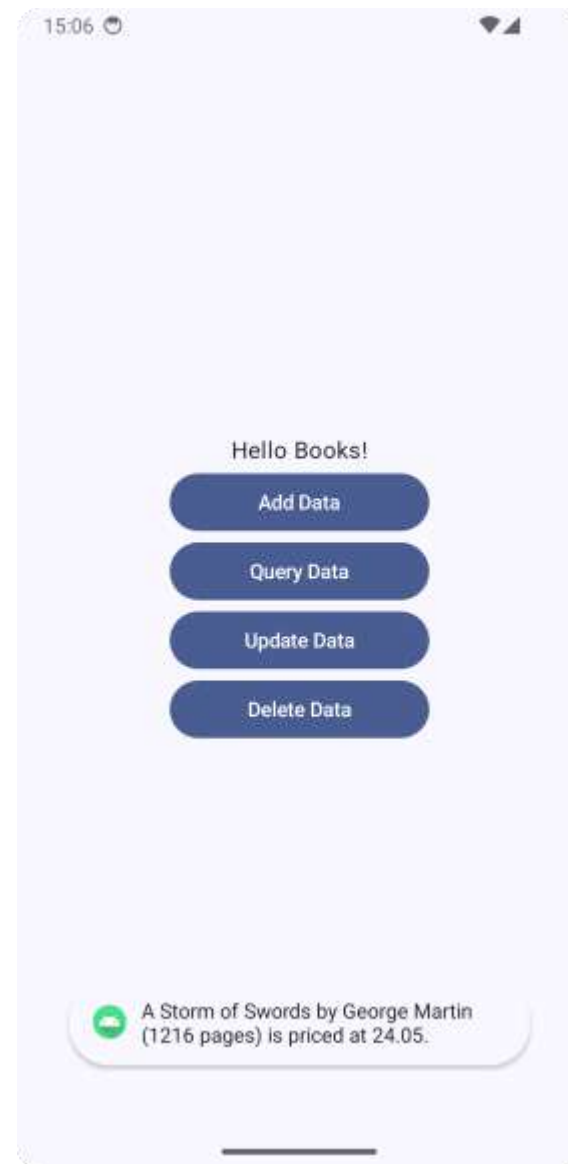
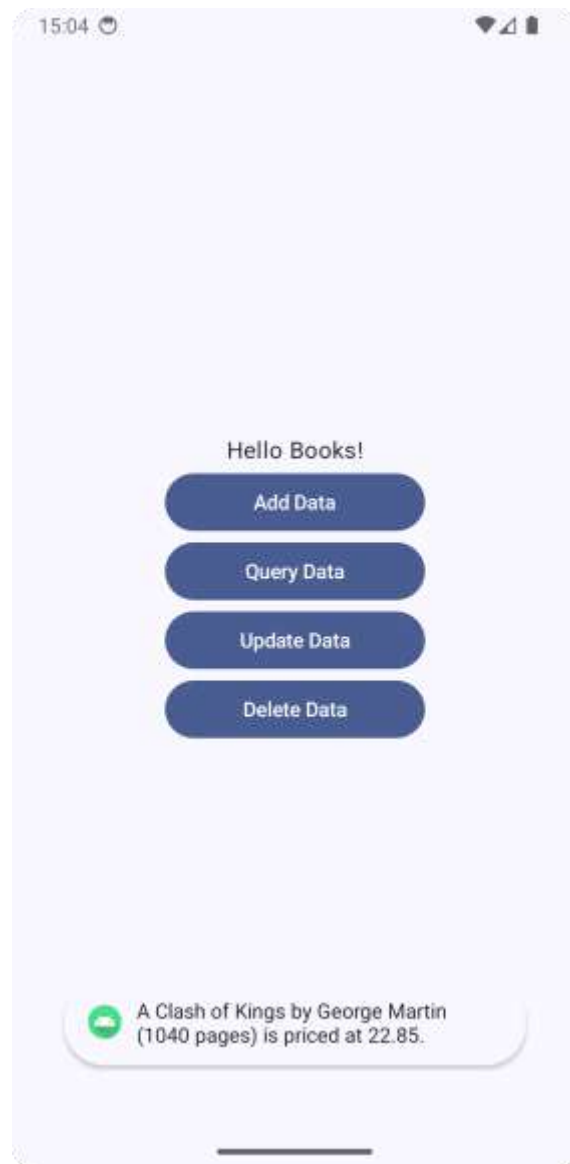
```
private fun addData(context: Context): String? {
    val uri = Uri.parse("content://com.example.cdatabasetest.provider/book")
    val values = ContentValuesOf(
        "name" to "A Clash of Kings",
        "author" to "George Martin",
        "pages" to 1040,
        "price" to 22.85
    )
    val newUri = context.contentResolver.insert(uri, values)
    return newUri?.pathSegments?.get(1)
}

private fun queryData(context: Context) {
    val uri = Uri.parse("content://com.example.cdatabasetest.provider/book")
    context.contentResolver.query(uri, null, null, null, null)?.apply {
        while (moveToNext()) {
            val bName = getString(getColumnIndexOrThrow("name"))
            val bAuthor = getString(getColumnIndexOrThrow("author"))
            val bPages = getInt(getColumnIndexOrThrow("pages"))
            val bPrice = getDouble(getColumnIndexOrThrow("price"))
            Toast.makeText(
                context,
                "$bName by $bAuthor ($bPages pages) is priced at $bPrice.",
                Toast.LENGTH_LONG
            ).show()
        }
        close()
    }
}
```

4.2 跨程序数据共享

```
private fun updateData(bookId: String?, context: Context) {  
    bookId?.let {  
        val uri = Uri.parse("content://com.example.cdbasetest.provider/book/$it")  
        val values = contentValuesOf("name" to "A Storm of Swords",  
            "pages" to 1216, "price" to 24.05)  
        context.contentResolver.update(uri, values, null, null)  
    }  
}  
  
private fun deleteData(bookId: String?, context: Context) {  
    bookId?.let {  
        val uri = Uri.parse("content://com.example.cdbasetest.provider/book/$it")  
        context.contentResolver.delete(uri, null, null)  
    }  
}
```


4.2 跨程序数据共享



感谢观看！